Flask Backend and REST API (Code)

```
from flask import Flask, request, jsonify
import sqlite3
from datetime import datetime

app = Flask(__name__)

DATABASE = "dripguard.db"


# ============================================================
# DATABASE
# ============================================================

def get_db():
    conn = sqlite3.connect(DATABASE)
    conn.row_factory = sqlite3.Row
    return conn


def init_database():
    conn = get_db()

    conn.execute("""
        CREATE TABLE IF NOT EXISTS readings (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            bed_id TEXT NOT NULL,
            drip_rate REAL NOT NULL,
            flow_rate REAL DEFAULT 0,
            bag_volume REAL DEFAULT 500,
```

```
            remaining_time REAL DEFAULT 0,
            status TEXT NOT NULL,
            timestamp TEXT NOT NULL
        )
    """)

    conn.commit()
    conn.close()


# ============================================================
# STATUS CALCULATION
# ============================================================

def calculate_status(drip_rate):

    if drip_rate <= 0:
        return "CRITICAL"

    elif drip_rate < 12:
        return "ATTENTION"

    else:
        return "NORMAL"


# ============================================================
# HOME
# ============================================================

@app.route("/")
```

```
def home():

    return jsonify({
        "system": "DripGuard",
        "status": "Backend running",
        "message": "Flask server is working"
    })


# ============================================================
# RECEIVE ESP8266 READING
# ============================================================

@app.route("/api/readings", methods=["POST"])
def receive_reading():

    try:

        data = request.get_json()

        if not data:
            return jsonify({
                "success": False,
                "error": "No JSON data received"
            }), 400

        bed_id = str(data.get("bed_id", "B01"))

        drip_rate = float(
            data.get("drip_rate", 0)
        )
```

```
flow_rate = float(
    data.get("flow_rate", drip_rate * 0.05)
)

bag_volume = float(
    data.get("bag_volume", 500)
)

# Calculate remaining time
if flow_rate > 0:
    remaining_time = bag_volume / flow_rate
else:
    remaining_time = 0

status = calculate_status(drip_rate)

timestamp = datetime.now().strftime(
    "%Y-%m-%d %H:%M:%S"
)

conn = get_db()

conn.execute("""
    INSERT INTO readings
    (
        bed_id,
        drip_rate,
        flow_rate,
        bag_volume,
        remaining_time,
```

```
            status,
            timestamp
        )
        VALUES (?, ?, ?, ?, ?, ?, ?)
    """, (
        bed_id,
        drip_rate,
        flow_rate,
        bag_volume,
        remaining_time,
        status,
        timestamp
    ))

    conn.commit()
    conn.close()

    return jsonify({
        "success": True,
        "bed_id": bed_id,
        "drip_rate": drip_rate,
        "flow_rate": flow_rate,
        "status": status,
        "remaining_time": remaining_time,
        "timestamp": timestamp
    })

except Exception as e:

    return jsonify({
        "success": False,
```

```
            "error": str(e)
        }), 500


# ============================================================
# GET LATEST READING FOR EACH BED
# ============================================================

@app.route("/api/latest", methods=["GET"])
def latest_readings():

    conn = get_db()

    rows = conn.execute("""
        SELECT *
        FROM readings
        WHERE id IN (
            SELECT MAX(id)
            FROM readings
            GROUP BY bed_id
        )
        ORDER BY bed_id
    """).fetchall()

    conn.close()

    readings = []

    for row in rows:

        readings.append({
```

```
            "id": row["id"],
            "bed_id": row["bed_id"],
            "drip_rate": row["drip_rate"],
            "flow_rate": row["flow_rate"],
            "bag_volume": row["bag_volume"],
            "remaining_time": row["remaining_time"],
            "status": row["status"],
            "timestamp": row["timestamp"]
        })

    return jsonify(readings)


# ============================================================
# GET HISTORY
# ============================================================

@app.route("/api/history", methods=["GET"])
def history():

    conn = get_db()

    rows = conn.execute("""
        SELECT *
        FROM readings
        ORDER BY id DESC
        LIMIT 100
    """).fetchall()

    conn.close()
```

```
    readings = []

    for row in rows:

        readings.append({
            "id": row["id"],
            "bed_id": row["bed_id"],
            "drip_rate": row["drip_rate"],
            "flow_rate": row["flow_rate"],
            "bag_volume": row["bag_volume"],
            "remaining_time": row["remaining_time"],
            "status": row["status"],
            "timestamp": row["timestamp"]
        })

    return jsonify(readings)


# ============================================================
# GET ALERTS
# ============================================================

@app.route("/api/alerts", methods=["GET"])
def alerts():

    conn = get_db()

    rows = conn.execute("""
        SELECT *
        FROM readings
        WHERE status != 'NORMAL'
```

```
        ORDER BY id DESC
        LIMIT 20
    """).fetchall()

    conn.close()

    alerts = []

    for row in rows:

        alerts.append({
            "id": row["id"],
            "bed_id": row["bed_id"],
            "drip_rate": row["drip_rate"],
            "status": row["status"],
            "timestamp": row["timestamp"]
        })

    return jsonify(alerts)


# ============================================================
# TEST DATA
# ============================================================

@app.route("/api/test", methods=["POST"])
def test_data():

    test_readings = [
        {
            "bed_id": "B01",
```

```
        "drip_rate": 20
    },
    {
        "bed_id": "B02",
        "drip_rate": 24
    },
    {
        "bed_id": "B03",
        "drip_rate": 9
    },
    {
        "bed_id": "B04",
        "drip_rate": 0
    }
]

results = []

conn = get_db()

for item in test_readings:

    bed_id = item["bed_id"]
    drip_rate = item["drip_rate"]

    flow_rate = drip_rate * 0.05

    bag_volume = 500

    if flow_rate > 0:
        remaining_time = bag_volume / flow_rate
```

```
else:
    remaining_time = 0

status = calculate_status(drip_rate)

timestamp = datetime.now().strftime(
    "%Y-%m-%d %H:%M:%S"
)

conn.execute("""
    INSERT INTO readings
    (
        bed_id,
        drip_rate,
        flow_rate,
        bag_volume,
        remaining_time,
        status,
        timestamp
    )
    VALUES (?, ?, ?, ?, ?, ?, ?)
""", (
    bed_id,
    drip_rate,
    flow_rate,
    bag_volume,
    remaining_time,
    status,
    timestamp
))
```

```
        results.append({
            "bed_id": bed_id,
            "drip_rate": drip_rate,
            "flow_rate": flow_rate,
            "status": status
        })

    conn.commit()
    conn.close()

    return jsonify({
        "success": True,
        "message": "Test readings inserted",
        "readings": results
    })


# ============================================================
# START SERVER
# ============================================================

if __name__ == "__main__":

    init_database()

    print("----------------------------------------")
    print("DripGuard Flask Backend")
    print("----------------------------------------")
    print("Server: http://127.0.0.1:5000")
    print("Latest: http://127.0.0.1:5000/api/latest")
    print("History: http://127.0.0.1:5000/api/history")
```

```
    print("Alerts: http://127.0.0.1:5000/api/alerts")
    print("-------------------------------------")

    app.run(
        host="0.0.0.0",
        port=5000,
        debug=True
    )
```

Streamlit Monitoring Dashboard (Code)

```
import streamlit as st
import sqlite3
import pandas as pd
from datetime import datetime

# ============================================================
# PAGE CONFIGURATION
# ============================================================

st.set_page_config(
    page_title="DripGuard",
    page_icon="💧",
    layout="wide",
    initial_sidebar_state="expanded"
)

DB_NAME = "dripguard.db"


# ============================================================
```

```
# CUSTOM CSS
# ============================================================

st.markdown("""
<style>

.stApp {
    background:
        radial-gradient(circle at top right,
        #092c55 0%,
        #03152d 35%,
        #020b18 100%);
    color: white;
}

header {
    background: transparent !important;
}

section[data-testid="stSidebar"] {
    background:
        linear-gradient(
            180deg,
            #061c39 0%,
            #031329 100%
        );
    border-right: 1px solid #12365c;
}

section[data-testid="stSidebar"] * {
    color: #dcecff;
```

```
}

.main-title {
    font-size: 42px;
    font-weight: 800;
    color: white;
    margin-bottom: 0;
}

.subtitle {
    font-size: 18px;
    color: #9bb9df;
    margin-top: 0;
}

.top-status {
    background: #062b2b;
    border: 1px solid #0bd99a;
    color: #19e6a4;
    padding: 8px 18px;
    border-radius: 20px;
    text-align: center;
    font-weight: 600;
}

.metric-card {
    background:
        linear-gradient(
            145deg,
            rgba(9, 47, 88, 0.95),
            rgba(4, 25, 51, 0.95)
```

```
        );
    border: 1px solid #174a7c;
    border-radius: 14px;
    padding: 20px;
    min-height: 135px;
    box-shadow: 0 8px 25px rgba(0,0,0,0.25);
}

.metric-title {
    color: #9dbbe0;
    font-size: 16px;
}

.metric-number {
    color: white;
    font-size: 35px;
    font-weight: 800;
    margin-top: 5px;
}

.metric-sub {
    color: #8aa8cc;
    font-size: 14px;
}

.normal-card {
    border: 1px solid #00e09a;
}

.attention-card {
    border: 1px solid #f5b82e;
```

```
}

.critical-card {
    border: 1px solid #ff334c;
}

.bed-card {
    background:
        linear-gradient(
            145deg,
            #082b50,
            #04182f
        );
    border-radius: 14px;
    padding: 18px;
    min-height: 280px;
}

.bed-normal {
    border: 2px solid #00e09a;
}

.bed-attention {
    border: 2px solid #f5b82e;
}

.bed-critical {
    border: 2px solid #ff334c;
}

.bed-title {
```

```
    font-size: 22px;
    font-weight: 800;
    color: white;
}

.status-normal {
    background: #08a968;
    color: white;
    padding: 5px 14px;
    border-radius: 20px;
    font-weight: 700;
    font-size: 13px;
}

.status-attention {
    background: #e9aa1b;
    color: #101010;
    padding: 5px 14px;
    border-radius: 20px;
    font-weight: 700;
    font-size: 13px;
}

.status-critical {
    background: #e82e45;
    color: white;
    padding: 5px 14px;
    border-radius: 20px;
    font-weight: 700;
    font-size: 13px;
}
```

```
.info-row {
    border-bottom: 1px solid #12385e;
    padding: 10px 0;
    color: #b9d0ec;
}

.info-value {
    float: right;
    color: white;
    font-weight: 700;
}

.section-box {
    background:
        linear-gradient(
            145deg,
            rgba(7, 37, 72, .95),
            rgba(3, 20, 40, .95)
        );
    border: 1px solid #164878;
    border-radius: 14px;
    padding: 20px;
}

.alert-critical {
    background: #351421;
    border-left: 5px solid #ff334c;
    padding: 12px;
    border-radius: 8px;
    margin-bottom: 10px;
```

```
}

.alert-attention {
    background: #342b13;
    border-left: 5px solid #f5b82e;
    padding: 12px;
    border-radius: 8px;
    margin-bottom: 10px;
}

.alert-normal {
    background: #0b3028;
    border-left: 5px solid #00e09a;
    padding: 12px;
    border-radius: 8px;
    margin-bottom: 10px;
}

.footer {
    text-align: center;
    color: #6f8eaf;
    margin-top: 30px;
}

</style>
""", unsafe_allow_html=True)


# ============================================================
# DATABASE
# ============================================================
```

```
def get_readings():

    conn = sqlite3.connect(DB_NAME)

    query = """
        SELECT
            id,
            bed_id,
            drops_per_minute,
            timestamp
        FROM readings
        ORDER BY id ASC
    """

    df = pd.read_sql_query(query, conn)

    conn.close()

    return df


# ============================================================
# STATUS
# ============================================================

def get_status(drops):

    drops = float(drops)

    if drops == 0:
```

```
        return "CRITICAL"

    elif drops < 10:
        return "ATTENTION"

    else:
        return "NORMAL"


# ============================================================
# SIDEBAR
# ============================================================

with st.sidebar:

    st.markdown("""
    <div style="
        text-align:center;
        padding:15px;
        font-size:25px;
        font-weight:800;
        color:white;">
        💧 Drip<span style="color:#159cff;">Guard</span>
    </div>
    """, unsafe_allow_html=True)

    st.markdown("---")

    page = st.radio(
        "",
        [
```

```
            "🏠 Dashboard",
            "📈 Live Monitoring",
            "🕘 History",
            "🔔 Alerts",
            "🗄️ Database",
            "⚙️ Settings"
        ]
    )

    st.markdown("---")

    st.markdown("""
    <div style="
        text-align:center;
        color:#7899bd;
        margin-top:180px;">
        ❤️<br>
        Better Monitoring.<br>
        Safer Care.
    </div>
    """, unsafe_allow_html=True)


# ============================================================
# LOAD DATA
# ============================================================

try:

    df = get_readings()
```

```
except Exception as e:

    st.error(f"Database error: {e}")
    st.stop()


# ============================================================
# HEADER
# ============================================================

col1, col2 = st.columns([4, 1])

with col1:

    st.markdown(
        '<div class="main-title">💧 DripGuard</div>',
        unsafe_allow_html=True
    )

    st.markdown(
        '<div class="subtitle">'
        'Smart IV Infusion Monitoring System'
        '</div>',
        unsafe_allow_html=True
    )

with col2:

    st.markdown(
        '<div class="top-status">🟢 Connected</div>',
        unsafe_allow_html=True
```

```
    )


st.divider()


# ============================================================
# DASHBOARD PAGE
# ============================================================

if page == "🏠 Dashboard":

    st.markdown(
        '<h1 style="font-size:32px;">IV Infusion Dashboard</h1>',
        unsafe_allow_html=True
    )

    st.markdown(
        '<p style="color:#9bb9df;font-size:17px;">'
        'Real-time monitoring of multiple IV units'
        '</p>',
        unsafe_allow_html=True
    )


    # ------------------------------------------------
    # NO DATA
    # ------------------------------------------------

    if df.empty:
```

```
    st.warning(
        "No ESP8266 readings have been received yet."
    )

    st.info(
        "Connect your ESP8266 and send readings "
        "to the Flask server."
    )

    st.stop()


# ----------------------------------------------------
# LATEST BED DATA
# ----------------------------------------------------

latest = (
    df.sort_values("timestamp")
      .groupby("bed_id")
      .tail(1)
      .copy()
)

latest["status"] = latest[
    "drops_per_minute"
].apply(get_status)


total = len(latest)

normal = len(
```

```
    latest[latest["status"] == "NORMAL"]
)

attention = len(
    latest[latest["status"] == "ATTENTION"]
)

critical = len(
    latest[latest["status"] == "CRITICAL"]
)


# ----------------------------------------------------
# TOP METRICS
# ----------------------------------------------------

c1, c2, c3, c4 = st.columns(4)

with c1:

    st.markdown(f"""
    <div class="metric-card">

    <div class="metric-title">
    💧 Total IVs
    </div>

    <div class="metric-number">
    {total}
    </div>
```

```
    <div class="metric-sub">
    Beds Monitored
    </div>

    </div>
    """, unsafe_allow_html=True)


with c2:

    st.markdown(f"""
    <div class="metric-card normal-card">

    <div class="metric-title"
    style="color:#00e09a;">
    ✔ Normal
    </div>

    <div class="metric-number">
    {normal}
    </div>

    <div class="metric-sub">
    IVs
    </div>

    </div>
    """, unsafe_allow_html=True)


with c3:
```

```
    st.markdown(f"""
    <div class="metric-card attention-card">

    <div class="metric-title"
    style="color:#f5b82e;">
    ⚠ Attention
    </div>

    <div class="metric-number">
    {attention}
    </div>

    <div class="metric-sub">
    IVs
    </div>

    </div>
    """, unsafe_allow_html=True)


with c4:

    st.markdown(f"""
    <div class="metric-card critical-card">

    <div class="metric-title"
    style="color:#ff334c;">
    🚨 Critical
    </div>
```

```
    <div class="metric-number">
    {critical}
    </div>

    <div class="metric-sub">
    IVs
    </div>

    </div>
    """, unsafe_allow_html=True)


st.markdown("### 🛏️ IV Bed Status")


# ---------------------------------------------------
# BED CARDS
# ---------------------------------------------------

bed_columns = st.columns(4)

for index, (_, row) in enumerate(
    latest.iterrows()
):

    bed = row["bed_id"]

    drops = float(
        row["drops_per_minute"]
    )
```

```
status = row["status"]


if status == "NORMAL":

    card_class = "bed-normal"
    status_class = "status-normal"
    status_text = "NORMAL"
    message = "🟢 Flowing Normally"

elif status == "ATTENTION":

    card_class = "bed-attention"
    status_class = "status-attention"
    status_text = "ATTENTION"
    message = "🟡 Low Drip Rate"

else:

    card_class = "bed-critical"
    status_class = "status-critical"
    status_text = "CRITICAL"
    message = "🔴 No Drops Detected"


# Approximate flow rate
flow_rate = drops * 0.05

# Example remaining time calculation
bag_volume = 500
```

```
if flow_rate > 0:

    remaining = bag_volume / flow_rate

    remaining_text = (
        f"{remaining:.0f} min"
    )

else:

    remaining_text = "STOPPED"


with bed_columns[index]:

    st.markdown(f"""

    <div class="bed-card {card_class}">

        <div style="
            display:flex;
            justify-content:space-between;
            align-items:center;">

            <div class="bed-title">
            🛏️ {bed}
            </div>

            <span class="{status_class}">
            {status_text}
            </span>
```

```
</div>

<div class="info-row">
    💧 Drip Rate
    <span class="info-value">
    {drops:.0f} drops/min
    </span>
</div>

<div class="info-row">
    ≋ Flow Rate
    <span class="info-value">
    {flow_rate:.2f} mL/min
    </span>
</div>

<div class="info-row">
    ⏱ Remaining Time
    <span class="info-value">
    {remaining_text}
    </span>
</div>

<div class="info-row">
    💉 Bag Volume
    <span class="info-value">
    500 mL
    </span>
</div>
```

```
            <br>

            <b>{message}</b>

        </div>

        """, unsafe_allow_html=True)


st.markdown("###")


# ----------------------------------------------------
# BOTTOM SECTION
# ----------------------------------------------------

chart_col, table_col, alert_col = st.columns(
    [1.4, 1.1, 1]
)


# ----------------------------------------------------
# DRIP RATE GRAPH
# ----------------------------------------------------

with chart_col:

    st.markdown(
        '<div class="section-box">',
        unsafe_allow_html=True
    )
```

```
st.markdown(
    "### 📈 Drip Rate Trend"
)

st.caption(
    "Live readings from ESP8266"
)

chart_df = df.copy()

chart_df["timestamp"] = pd.to_datetime(
    chart_df["timestamp"]
)

chart_df = chart_df.pivot_table(
    index="timestamp",
    columns="bed_id",
    values="drops_per_minute",
    aggfunc="last"
)

st.line_chart(
    chart_df,
    height=300
)

st.markdown(
    "</div>",
    unsafe_allow_html=True
)
```

```
# ----------------------------------------------------
# RECENT READINGS
# ----------------------------------------------------

with table_col:

    st.markdown(
        '<div class="section-box">',
        unsafe_allow_html=True
    )

    st.markdown(
        "### 📋 Recent Readings"
    )

    recent = df.tail(8).copy()

    recent["Status"] = recent[
        "drops_per_minute"
    ].apply(get_status)

    recent = recent.rename(
        columns={
            "bed_id": "Bed",
            "drops_per_minute": "Drops/min",
            "timestamp": "Time"
        }
    )
```

```
    st.dataframe(
        recent[
            [
                "Time",
                "Bed",
                "Drops/min",
                "Status"
            ]
        ].iloc[::-1],
        use_container_width=True,
        hide_index=True
    )

    st.markdown(
        "</div>",
        unsafe_allow_html=True
    )


# ----------------------------------------------------
# ALERTS
# ----------------------------------------------------

with alert_col:

    st.markdown(
        '<div class="section-box">',
        unsafe_allow_html=True
    )

    st.markdown(
```

```
    "### 🔔 Latest Alerts"
)

alert_data = latest[
    latest["status"] != "NORMAL"
].sort_values(
    "timestamp",
    ascending=False
)

if alert_data.empty:

    st.success(
        "No active alerts."
    )

else:

    for _, alert in alert_data.iterrows():

        bed = alert["bed_id"]

        drops = float(
            alert["drops_per_minute"]
        )

        status = alert["status"]

        time = alert["timestamp"]

        if status == "CRITICAL":
```

```
                st.markdown(f"""
                <div class="alert-critical">
                <b>🚨 {bed} - Critical</b><br>
                No drops detected.<br>
                <small>{time}</small>
                </div>
                """,
                unsafe_allow_html=True)

            else:

                st.markdown(f"""
                <div class="alert-attention">
                <b>⚠️ {bed} - Attention</b><br>
                Low drip rate
                ({drops:.0f} drops/min).<br>
                <small>{time}</small>
                </div>
                """,
                unsafe_allow_html=True)


        st.markdown(
            "</div>",
            unsafe_allow_html=True
        )


# ============================================================
# LIVE MONITORING
```

```
# ========================================================

elif page == "📈 Live Monitoring":

    st.title("📈 Live Monitoring")

    if df.empty:

        st.warning("No ESP8266 data available.")

    else:

        latest = (
            df.sort_values("timestamp")
              .groupby("bed_id")
              .tail(1)
        )

        st.dataframe(
            latest[
                [
                    "bed_id",
                    "drops_per_minute",
                    "timestamp"
                ]
            ],
            use_container_width=True,
            hide_index=True
        )
```

```
# ============================================================
# HISTORY
# ============================================================

elif page == "🕘 History":

    st.title("🕘 Reading History")

    if df.empty:

        st.info("No historical readings yet.")

    else:

        st.dataframe(
            df.sort_values(
                "timestamp",
                ascending=False
            ),
            use_container_width=True,
            hide_index=True
        )


# ============================================================
# ALERTS
# ============================================================

elif page == "🔔 Alerts":

    st.title("🔔 Alerts")
```

```
if df.empty:

    st.info("No alerts.")

else:

    alerts = df[
        df["drops_per_minute"] < 10
    ].copy()

    if alerts.empty:

        st.success(
            "No critical or attention readings."
        )

    else:

        alerts["Status"] = alerts[
            "drops_per_minute"
        ].apply(get_status)

        st.dataframe(
            alerts.sort_values(
                "timestamp",
                ascending=False
            ),
            use_container_width=True,
            hide_index=True
        )
```

```
# ============================================================
# DATABASE
# ============================================================

elif page == "🗄 Database":

    st.title("🗄 Database")

    st.write(
        f"Database: `{DB_NAME}`"
    )

    st.metric(
        "Total Stored Readings",
        len(df)
    )

    if not df.empty:

        st.dataframe(
            df,
            use_container_width=True,
            hide_index=True
        )


# ============================================================
# SETTINGS
# ============================================================
```

```
elif page == "⚙️ Settings":

    st.title("⚙️ Settings")

    st.write(
        "DripGuard System Configuration"
    )

    st.text_input(
        "Hospital / Project Name",
        "DripGuard"
    )

    st.number_input(
        "Warning threshold (drops/min)",
        min_value=1,
        max_value=50,
        value=10
    )

    st.number_input(
        "Critical threshold (drops/min)",
        min_value=0,
        max_value=10,
        value=0
    )

    st.success(
        "Settings page ready."
    )
```

```
# ============================================================
# FOOTER
# ============================================================

st.markdown(
    """
    <div class="footer">
    ❤️ Better Monitoring. Safer Care. |
    DripGuard Smart IV Infusion Monitoring System
    </div>
    """,
    unsafe_allow_html=True
)
```

Streamlit Demo Dashboard (Code)

```
import streamlit as st
import pandas as pd
import numpy as np
from datetime import datetime, timedelta

# ============================================================
# PAGE CONFIGURATION
# ============================================================

st.set_page_config(
    page_title="DripGuard",
    page_icon=" 💧 ",
    layout="wide",
```

```
    initial_sidebar_state="expanded"
)

# ============================================================
# CUSTOM CSS
# ============================================================

st.markdown("""
<style>

/* MAIN BACKGROUND */
.stApp {
    background-color: #061a33;
    color: white;
}

/* Remove top Streamlit padding */
.block-container {
    padding-top: 1.5rem;
    padding-left: 2rem;
    padding-right: 2rem;
    max-width: 1600px;
}

/* SIDEBAR */
section[data-testid="stSidebar"] {
    background-color: #071d36;
    border-right: 1px solid #163a60;
}

section[data-testid="stSidebar"] > div {
```

```
    padding-top: 2rem;
}

section[data-testid="stSidebar"] p,
section[data-testid="stSidebar"] span,
section[data-testid="stSidebar"] label {
    color: #d9e7f7 !important;
}

/* TITLES */
.main-title {
    font-size: 42px;
    font-weight: 800;
    color: white;
    margin-bottom: 3px;
}

.subtitle {
    font-size: 18px;
    color: #9bbce2;
    margin-bottom: 20px;
}

.section-title {
    font-size: 25px;
    font-weight: 750;
    color: white;
    margin-top: 25px;
    margin-bottom: 5px;
}
```

```
/* CONNECTION STATUS */
.connected {
    background-color: #063d35;
    color: #31e69b;
    border: 1px solid #16d99a;
    border-radius: 30px;
    padding: 12px 28px;
    text-align: center;
    font-size: 17px;
    font-weight: 700;
}

/* METRIC CARDS */
.metric-card {
    background-color: #0b294d;
    border: 1px solid #194a76;
    border-radius: 12px;
    padding: 18px;
    min-height: 125px;
}

.metric-title {
    color: #91b5dd;
    font-size: 16px;
    margin-bottom: 5px;
}

.metric-number {
    color: white;
    font-size: 34px;
    font-weight: 800;
```

```
}

.metric-small {
    color: #a6c3e2;
    font-size: 14px;
}

.metric-normal { color: #28e39a; }
.metric-attention { color: #ffc02e; }
.metric-critical { color: #ff4554; }

/* IV CARDS */
.iv-card {
    background-color: #0b294d;
    border-radius: 13px;
    padding: 18px;
    min-height: 335px;
    border: 2px solid #173e65;
}

.iv-normal { border-color: #16d99a; }
.iv-attention { border-color: #ffbf24; }
.iv-critical { border-color: #ff4051; }

.iv-title {
    color: white;
    font-size: 21px;
    font-weight: 750;
}

.iv-status-badge {
```

```
    border-radius: 20px;
    padding: 5px 12px;
    font-size: 13px;
    font-weight: 700;
    color: white;
}

.bg-normal { background-color: #0b9f62; }
.bg-attention { background-color: #e5a817; }
.bg-critical { background-color: #df3344; }

.flex-row {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-top: 14px;
}

.data-label {
    color: #a5c1df;
    font-size: 14px;
}

.data-value {
    color: white;
    font-size: 15px;
    font-weight: 700;
}

.status-normal { color: #25e39a; font-weight: 700; }
.status-attention { color: #ffc12d; font-weight: 700; }
```

```
.status-critical { color: #ff4657; font-weight: 700; }

/* PANELS & ALERTS */
.panel {
    background-color: #0b294d;
    border: 1px solid #194a76;
    border-radius: 12px;
    padding: 18px;
}

.panel-title {
    font-size: 20px;
    font-weight: 750;
    color: white;
}

.alert-normal {
    background-color: #073b31;
    border-left: 5px solid #22d894;
    padding: 12px;
    border-radius: 7px;
    margin-bottom: 8px;
}

.alert-attention {
    background-color: #47370a;
    border-left: 5px solid #ffc32b;
    padding: 12px;
    border-radius: 7px;
    margin-bottom: 8px;
}
```

```
.alert-critical {
    background-color: #48151d;
    border-left: 5px solid #ff4354;
    padding: 12px;
    border-radius: 7px;
    margin-bottom: 8px;
}

.sidebar-brand {
    font-size: 28px;
    font-weight: 800;
    text-align: center;
    margin-bottom: 25px;
}

.brand-blue { color: #239cff; }

.footer {
    text-align: center;
    color: #7fa2c9;
    padding-top: 25px;
    padding-bottom: 20px;
}

</style>
""", unsafe_allow_html=True)


# ============================================================
# SAMPLE DATA
```

```
# ============================================================

beds = [
    {
        "bed": "B01",
        "drip": 20,
        "flow": 1.00,
        "remaining": "480 min (8.0 hrs)",
        "volume": 500,
        "status": "NORMAL",
        "message": "Flowing Normally"
    },
    {
        "bed": "B02",
        "drip": 24,
        "flow": 1.20,
        "remaining": "417 min (6.9 hrs)",
        "volume": 500,
        "status": "NORMAL",
        "message": "Flowing Normally"
    },
    {
        "bed": "B03",
        "drip": 9,
        "flow": 0.45,
        "remaining": "1,111 min (18.5 hrs)",
        "volume": 500,
        "status": "ATTENTION",
        "message": "Low Drip Rate"
    },
    {
```

```
        "bed": "B04",
        "drip": 0,
        "flow": 0.00,
        "remaining": "STOPPED",
        "volume": 500,
        "status": "CRITICAL",
        "message": "No Drops Detected"
    }
]


# ============================================================
# SIDEBAR
# ============================================================

with st.sidebar:
    st.markdown(
        '<div class="sidebar-brand">💧 Drip<span class="brand-blue">Guard</span></div>',
        unsafe_allow_html=True
    )

    st.markdown("---")

    page = st.radio(
        "Navigation",
        [
            "🏠 Dashboard",
            "📈 Live Monitoring",
            "🕘 History",
            "🔔 Alerts",
            "🗄️ Database",
```

```
            "⚙️ Settings"
        ],
        label_visibility="collapsed"
    )

    st.markdown("---")

    st.markdown(
        """
        <div style="text-align:center; color:#7fa2c9; margin-top:220px;">
        ❤️<br><br>
        Better Monitoring.<br>
        Safer Care.
        </div>
        """,
        unsafe_allow_html=True
    )


# ============================================================
# HEADER
# ============================================================

header_col1, header_col2 = st.columns([4, 1])

with header_col1:
    st.markdown(
        '<div class="main-title">💧 Drip<span class="brand-blue">Guard</span></div>',
        unsafe_allow_html=True
    )
    st.markdown(
```

```
        '<div class="subtitle">Smart IV Infusion Monitoring System</div>',
        unsafe_allow_html=True
    )

with header_col2:
    st.markdown(
        '<div class="connected">🟢 Connected</div>',
        unsafe_allow_html=True
    )

st.markdown("---")


# ============================================================
# DASHBOARD TITLE
# ============================================================

st.markdown(
    '<div class="section-title">IV Infusion Dashboard</div>',
    unsafe_allow_html=True
)

st.markdown(
    '<div class="subtitle">Real-time monitoring of multiple IV units</div>',
    unsafe_allow_html=True
)


# ============================================================
# SUMMARY COUNTERS
# ============================================================
```

```
normal_count = sum(1 for x in beds if x["status"] == "NORMAL")
attention_count = sum(1 for x in beds if x["status"] == "ATTENTION")
critical_count = sum(1 for x in beds if x["status"] == "CRITICAL")

c1, c2, c3, c4 = st.columns(4)

with c1:
    st.markdown(
        """
        <div class="metric-card">
            <div class="metric-title">💧 Total IVs</div>
            <div class="metric-number">4</div>
            <div class="metric-small">Beds Monitored</div>
        </div>
        """,
        unsafe_allow_html=True
    )

with c2:
    st.markdown(
        f"""
        <div class="metric-card">
            <div class="metric-title metric-normal">✓ Normal</div>
            <div class="metric-number">{normal_count}</div>
            <div class="metric-small">IVs</div>
        </div>
        """,
        unsafe_allow_html=True
    )
```

```
with c3:
    st.markdown(
        f"""
        <div class="metric-card">
            <div class="metric-title metric-attention">⚠ Attention</div>
            <div class="metric-number">{attention_count}</div>
            <div class="metric-small">IVs</div>
        </div>
        """,
        unsafe_allow_html=True
    )

with c4:
    st.markdown(
        f"""
        <div class="metric-card">
            <div class="metric-title metric-critical">🚨 Critical</div>
            <div class="metric-number">{critical_count}</div>
            <div class="metric-small">IVs</div>
        </div>
        """,
        unsafe_allow_html=True
    )


# ============================================================
# IV BED STATUS
# ============================================================

st.markdown(
    '<div class="section-title">IV Bed Status</div>',
```

```
    unsafe_allow_html=True
)

bed_columns = st.columns(4)

for i, bed in enumerate(beds):

    with bed_columns[i]:

        if bed["status"] == "NORMAL":
            card_class = "iv-normal"
            bg_badge = "bg-normal"
            text_class = "status-normal"
        elif bed["status"] == "ATTENTION":
            card_class = "iv-attention"
            bg_badge = "bg-attention"
            text_class = "status-attention"
        else:
            card_class = "iv-critical"
            bg_badge = "bg-critical"
            text_class = "status-critical"

        st.markdown(
            f"""
            <div class="iv-card {card_class}">
                <div class="flex-row" style="margin-top:0; margin-bottom:15px;">
                    <div class="iv-title">🛏️ {bed["bed"]}</div>
                    <div class="iv-status-badge {bg_badge}">{bed["status"]}</div>
                </div>
                <hr style="border-color:#21496e; margin:0;">
                <div class="flex-row">
```

```
                <span class="data-label">Drip Rate</span>
                <span class="data-value">{bed["drip"]} drops/min</span>
            </div>
            <div class="flex-row">
                <span class="data-label">Flow Rate</span>
                <span class="data-value">{bed["flow"]:.2f} mL/min</span>
            </div>
            <div class="flex-row">
                <span class="data-label">Remaining Time</span>
                <span class="data-value">{bed["remaining"]}</span>
            </div>
            <div class="flex-row">
                <span class="data-label">Bag Volume</span>
                <span class="data-value">{bed["volume"]} mL</span>
            </div>
            <hr style="border-color:#21496e; margin-top:18px; margin-bottom:12px;">
            <div class="{text_class}">
                ● {bed["message"]}
            </div>
        </div>
        """,
        unsafe_allow_html=True
    )


# ============================================================
# GRAPH DATA
# ============================================================

times = [(datetime.now() - timedelta(minutes=9-i)).strftime("%H:%M") for i in range(10)]
```

```
graph_data = pd.DataFrame(
    {
        "Time": times,
        "B01": [19, 20, 21, 20, 20, 21, 20, 20, 20, 20],
        "B02": [22, 23, 22, 24, 23, 24, 24, 24, 24, 24],
        "B03": [10, 9, 9, 8, 9, 10, 9, 9, 9, 9],
        "B04": [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
    }
).set_index("Time")


# ============================================================
# BOTTOM SECTION
# ============================================================

left, middle, right = st.columns([1.5, 1, 1])

# DRIP RATE GRAPH
with left:
    st.markdown(
        """
        <div class="panel">
            <div class="panel-title">📈 Drip Rate Trend</div>
            <div style="color:#8faed0;">Last 10 Minutes</div>
        </div>
        """,
        unsafe_allow_html=True
    )
    st.line_chart(graph_data, height=280)

# RECENT READINGS
```

```
with middle:
    st.markdown(
        """
        <div class="panel">
            <div class="panel-title">📋 Recent Readings</div>
        </div>
        """,
        unsafe_allow_html=True
    )

    recent_data = pd.DataFrame(
        [
            ["14:31", "B04", 0, "CRITICAL"],
            ["14:30", "B03", 9, "ATTENTION"],
            ["14:29", "B02", 24, "NORMAL"],
            ["14:28", "B01", 20, "NORMAL"],
            ["14:27", "B04", 0, "CRITICAL"],
        ],
        columns=["Time", "Bed ID", "Drops/min", "Status"]
    )

    st.dataframe(
        recent_data,
        use_container_width=True,
        hide_index=True
    )

# LATEST ALERTS
with right:
    st.markdown(
        """
```

```
        <div class="panel">
            <div class="panel-title">🔔 Latest Alerts</div>
        </div>
        """,
        unsafe_allow_html=True
    )

    st.markdown(
        """
        <div class="alert-critical">
            <b>🚨 B04 - Critical</b><br>
            <small>No drops detected. Check IV line.</small>
        </div>

        <div class="alert-attention">
            <b>⚠️ B03 - Attention</b><br>
            <small>Low drip rate: 9 drops/min.</small>
        </div>

        <div class="alert-normal">
            <b>✓ B02 - Normal</b><br>
            <small>Drip rate back to normal.</small>
        </div>
        """,
        unsafe_allow_html=True
    )


# ============================================================
# FOOTER
# ============================================================
```

```
st.markdown(
    """
    <div class="footer">
        DripGuard • Smart IV Infusion Monitoring System
        <br>
        ESP8266 + IR Sensor + Python Dashboard
    </div>
    """,
    unsafe_allow_html=True
)
```